

Lixeira Inteligente

Descrição detalhada de funcionamento • versão 1.1.1+5

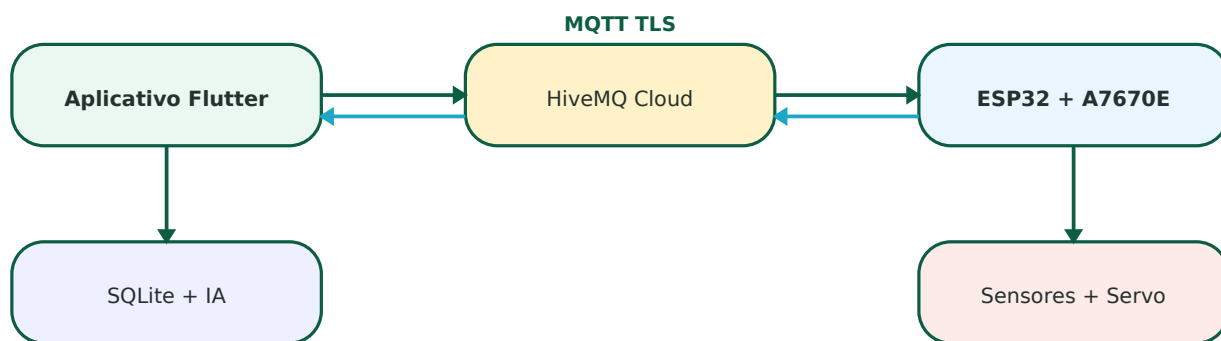
Sistema inteligente de coleta seletiva conectado à nuvem

Aplicativo	Flutter + Dart • Android/iOS • MQTT/HiveMQ • SQLite • Câmera + IA
Firmware	ESP32 / LilyGo T-Call A7670E • Wi-Fi + 4G • MQTT TLS • sensores + servo
Escola	Escola Técnica Estadual Aderico Alves de Vasconcelos
Equipe	Daniel Henrique da Costa Ferreira • Clarice Lima Souza • 3º A Redes
Local	Goiana - PE • 2026

Tecnologia aplicada à educação sustentável, monitoramento IoT e apoio à separação de materiais recicláveis.

1. Visão geral

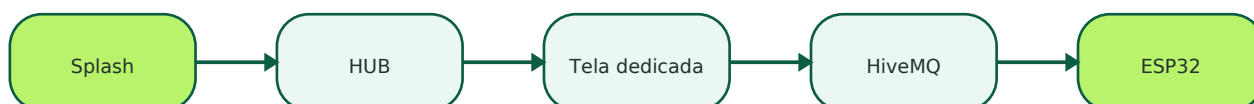
A Lixeira Inteligente integra sensores físicos, conectividade IoT, aplicativo móvel e visão computacional. O conceito escolar prevê uma única abertura de descarte, identificação do resíduo e separação automática. A versão 1.1.1+5 implementa a base funcional de monitoramento, classificação, telemetria e controle da comporta.



Arquitetura geral da implementação atual.

2. Jornada do usuário

- O usuário abre o app e entra diretamente na Splash personalizada com motion design.
- A Splash aguarda a inicialização do banco/configurações e permanece visível por no mínimo 3,6 segundos.
- O HUB mostra status HiveMQ, rede do ESP32, RSSI e seis leituras principais.
- Cada leitura abre uma tela dedicada com Lottie, estado e valor animado.
- Na aba IA, a câmera só é inicializada depois do botão Iniciar análise por câmera.
- A foto é combinada com evidências visuais e, quando presente, com o sensor indutivo.
- A aba Configurações concentra host, porta, usuário, senha, deviceId e tópicos MQTT.
- Com MQTT conectado, a tela da comporta permite Abrir/Fechar e o ESP32 responde com ACK.



Fluxo visual e operacional resumido.

3. Splash, identidade visual e motion design

A experiência visual foi redesenhada para evitar uma tela genérica antes da Splash. O Flutter é iniciado imediatamente; banco e configurações são carregados enquanto a Splash já está visível. Android e iOS recebem branding nativo coerente com o primeiro frame da experiência Flutter.

Token	Cor	Uso
Deep Green	#063E35	fundos escuros e identidade
Emerald	#13B77A	primária, conexão e sucesso
Mint	#EAF8F1	estados suaves
Cyan	#1DA7C6	peso e informações
Amber	#F3A712	nível/atenção
Violet	#7E57C2	presença e IA
Coral	#FF6B5E	comporta/erro/alerta

Elementos de motion

- Splash com gradiente animado, Lottie, fade, slide e scale.
- StaggeredReveal: entrada progressiva dos cards com opacidade, deslocamento e escala.
- PulseDot: status HiveMQ com pulsação.
- AnimatedSwitcher: transição entre as quatro áreas principais.
- AnimatedScale e Hero nas leituras do HUB.
- Lottie local para HUB, peso, nível, presença, metal, comporta, conectividade, IA, loading, sucesso e estado desconhecido.

4. Tela inicial - HUB

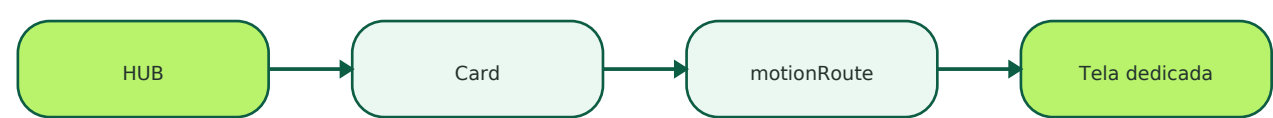
A tela inicial é exclusivamente um HUB de monitoramento. Não contém campos de conexão HiveMQ; ela apenas exibe o estado atual e encaminha o usuário às leituras especializadas.

Card	Valor exibido	Cor/semântica
Peso	gramas	Cyan
Nível	percentual	Amber
Presença	Objeto / Livre	Violet
Metal	Detectado / Não	cinza metálico
Comporta	OPEN / CLOSED	Coral
Rede	WIFI / 4G	Emerald

5. Telas dedicadas

Cada card do HUB abre uma tela própria usando uma rota com fade e slide. As telas usam o mesmo padrão visual: cabeçalho, Lottie, valor animado, estado interpretado e explicação do sensor.

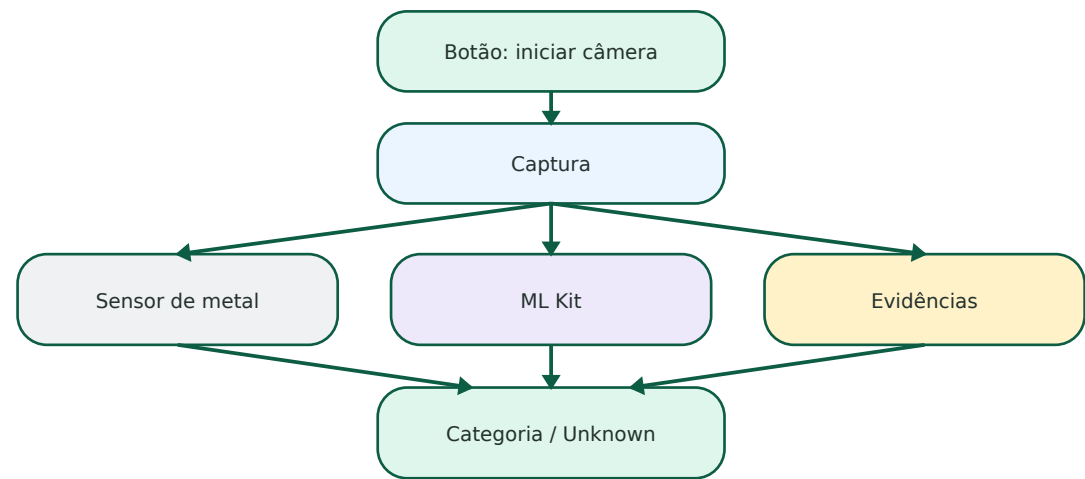
Tela	Interpretação
Peso	Objeto sobre a balança / aguardando objeto
Nível	Disponível / atenção / crítico a partir de 85%
Presença	Objeto detectado / área livre
Metal	Metal identificado / sem metal
Comporta	Estado atual + Abrir/Fechar quando MQTT está conectado
Conectividade	HiveMQ conectado/offline + interface e RSSI



Navegação das leituras.

6. IA e visão computacional

A aba IA não abre a câmera automaticamente. Primeiro apresenta uma landing visual, explica o processo e oferece o botão Iniciar análise por câmera. Só então o CameraController é criado. A captura usa ResolutionPreset.medium e não grava áudio.



Pipeline de classificação atual.

7. Classificação e tratamento de incerteza

O Image Labeling do ML Kit é usado como fonte de evidências. O projeto adota uma lista de palavras-chave por categoria e evita forçar classificação quando a evidência é insuficiente.

Categoria	Exemplos de palavras-chave
Papel	paper, cardboard, carton, newspaper, book, document, box
Plástico	plastic, bottle, container, packaging, jug, cup
Metal	metal, aluminum, aluminium, can, tin, steel
Vidro	glass, jar, wine bottle
Não identificado	sem evidência compatível ou confiança final < 60%

O ImageLabeler recebe rótulos a partir de 45%. As evidências compatíveis são somadas por categoria. Se o valor normalizado da melhor categoria ficar abaixo de 60%, a interface pede outra foto. Quando o sensor indutivo está ativo, metal é confirmado fisicamente com confiança 0,99.

8. MQTT / HiveMQ

Função	Tópico padrão	Direção
Telemetria	lixeira/telemetry	ESP32 -> app
Comando	lixeira/command	app -> ESP32
ACK	lixeira/ack	ESP32 -> app
Status	lixeira/status	ESP32 -> app

No aplicativo, o MQTT usa TLS, porta padrão 8883, QoS at least once, autoReconnect e resubscribeOnAutoReconnect. Quando tópicos separados são usados, o app também assina os tópicos irmãos /ack e /status.

Comando e ACK

```
{"type": "command", "deviceId": "LIXEIRA-01", "commandId": "uuid-v4", "action": "open"}
```

```
{"type": "ack", "deviceId": "LIXEIRA-01", "commandId": "uuid-v4", "action": "open", "status": "ok"}
```

9. Persistência local e segurança

O aplicativo usa SQLite local-first. A telemetria recebida é persistida e eventos de IA, comandos, ACKs e status são registrados em uma linha do tempo local.

Tabela	Conteúdo
telemetry	device_id, peso, metal, presença, nível, comporta, rede, RSSI e horário
events	kind, title, detail e created_at
settings	host, porta, usuário, tópicos, tópico único e deviceId

A senha MQTT não é armazenada no SQLite. Ela usa flutter_secure_storage, chave mqtt_password. O firmware distribuído também usa host, Wi-Fi e credenciais vazias em config.h, evitando incorporar segredos reais no ZIP.

10. Histórico

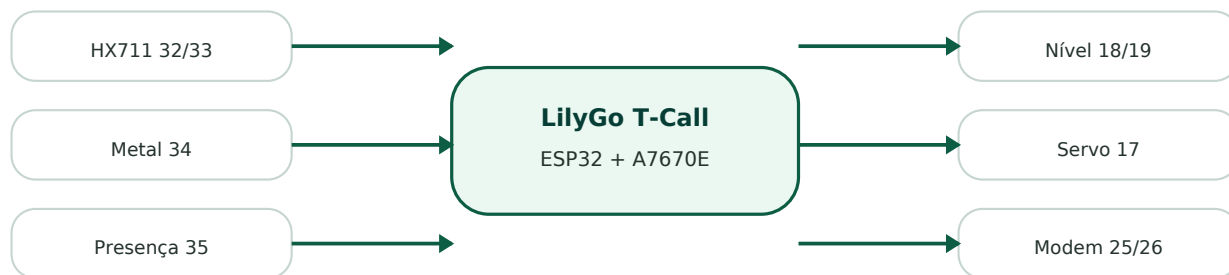
A aba Histórico apresenta eventos locais com animação de entrada, cores e ícones. Os filtros visuais atuais são Todos, IA, Comandos e Status.

11. Arquitetura do aplicativo

Camada/área	Responsabilidade	Exemplos
Presentation	telas, navegação e motion	DashboardPage, CameraAiPage, SensorDetailPage
Coordenação	estado global e fluxo MQTT	AppController
Domain	modelos puros	Telemetry, DeviceAck, MqttSettings, RecyclingResult
Infrastructure	rede, banco e segredos	MqttService, AppDatabase, SettingsRepository
Vision	adaptação ML Kit	VisionService

12. Firmware ESP32

O firmware é destinado à LilyGo TTGO T-Call A7670E V1.0. main.cpp coordena conectividade, MQTT, sensores, servo e telemetria. A mensagem é publicada a cada 5 segundos.



Mapa funcional de pinos.

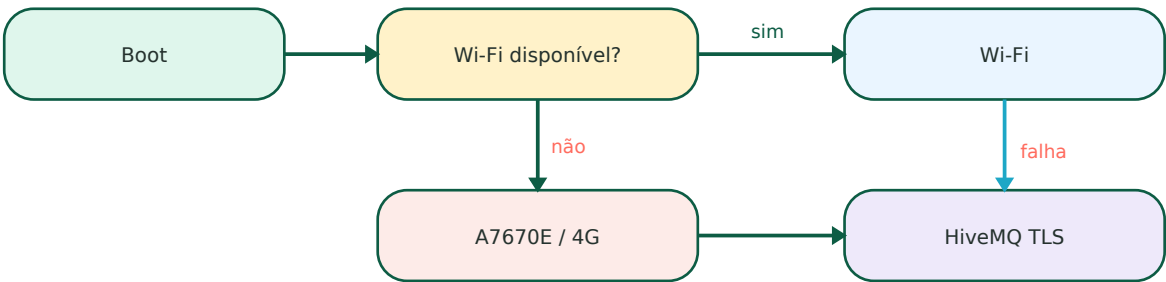
Função	GPIO
Modem TX / RX	26 / 25
Modem PWRKEY / RESET / DTR	4 / 27 / 14
HX711 DOUT / SCK	32 / 33
Metal	34
Presença	35
Nível TRIG / ECHO	18 / 19
Servo	17

13. Sensores e atuador

- HX711: tara no boot, cinco amostras por leitura e fator inicial -7050.0.
- Metal: entrada digital com possibilidade de lógica active-low.
- Presença: entrada digital com lógica configurável.
- Nível: distância por pulso e conversão linear entre 45 cm (vazia) e 5 cm (cheia).
- Servo: 15° fechado, 95° aberto e fechamento automático após 3,5 s.

14. Conectividade Wi-Fi / 4G

Wi-Fi é sempre a interface preferida. Quando não está disponível, o firmware inicializa o A7670E e tenta APNs pré-configurados para TIM, Claro e Vivo. Quando o Wi-Fi reaparece durante o 4G, o sistema exige estabilidade antes do failback.



Política de rede do firmware.

Parâmetro	Valor
Checagem de rede	10 s
Timeout Wi-Fi inicial	15 s
Estabilidade antes de voltar ao Wi-Fi	20 s
Reconexão MQTT	aprox. 5 s
Telemetria	5 s

15. MQTT seguro no ESP32

No Wi-Fi, o firmware usa WiFiClientSecure + PubSubClient com a CA ISRG Root X1. No 4G, usa o caminho MQTT/TLS do A7670E no fork TinyGSM, com SNI/certificado e autenticação do HiveMQ.

O status usa Last Will no caminho Wi-Fi. Comandos são assinados com QoS 1. O firmware aceita open, close e ping.

16. Tolerância a falhas

Mecanismo	Comportamento
Fila offline	até 20 telemetrias; ao lotar, remove a mais antiga
Flush	envia a fila quando MQTT volta
Deduplicação	cache circular dos 10 commandIds recentes
Failover	Wi-Fi -> 4G
Failback	4G -> Wi-Fi após 20 s estável
Servo	auto close após 3,5 s
MQTT	reconexão periódica e rebinding à interface ativa

17. Quick Start e ambiente

O projeto inclui um bootstrap Windows inspirado no fluxo de instalação automatizada. Ele procura Flutter/Dart, seleciona JDK compatível, executa flutter doctor, licenças Android, dependências, configuração nativa, análise estática e testes.

```
powershell -ExecutionPolicy Bypass -File .\ool\bootstrap_windows.ps1
```

Para validar e gerar APK debug no mesmo fluxo:

```
powershell -ExecutionPolicy Bypass -File .\ool\bootstrap_windows.ps1 -BuildApk
```

Requisito	Projeto
Flutter	>= 3.47.0
Dart	>= 3.12.0
JDK	17 <= versão < 26
Android minSdk	24
iOS mínimo	15.5
Application ID	br.edu.ete.lixeirainteligente

18. Build, APK e distribuição

O APK pode ser gerado diretamente pelo Flutter.

```
flutter build apk --release
```

Saída típica:

```
build/app/outputs/flutter-apk/app-release.apk
```

Para distribuição pública/loja, recomenda-se configurar assinatura própria e gerar AAB quando aplicável.

19. Testes e validação

O teste automatizado atual verifica parsing da telemetria. O bootstrap interrompe o processo quando flutter analyze ou flutter test falham, evitando uma falsa mensagem de sucesso.

- Telemetry.fromJson: deviceId, metalDetected e weightGrams.
- flutter analyze durante o bootstrap.
- flutter test durante o bootstrap.
- validação de Flutter/Dart/JDK antes das etapas de build.
- configure_platforms.dart reaplica permissões, IDs, minSdk, Java e branding.

20. Fluxos principais

Fluxo	Sequência
Monitoramento	sensores -> ESP32 -> HiveMQ -> app -> HUB -> SQLite
Comporta	app -> commandId -> MQTT -> ESP32 -> servo -> ACK -> histórico
IA	botão -> câmera -> ML Kit + metal -> categoria/unknown -> histórico
Falha Wi-Fi	Wi-Fi -> 4G -> reconectar MQTT -> flush -> fallback estável

21. Demonstração sugerida para a feira

- Abrir o aplicativo e demonstrar a Splash com identidade própria.
- Mostrar o HUB e explicar que HiveMQ é apenas indicado ali; a configuração fica na aba Config.
- Produzir telemetria real ou simulada e mostrar os cards mudando.
- Abrir Peso, Nível, Presença, Metal, Comporta e Rede.
- Ativar o sensor indutivo com um objeto metálico.
- Abrir IA, mostrar que a câmera não inicia sozinha e pressionar o botão.
- Capturar um resíduo e explicar categoria, confiança e evidências.
- Enviar Abrir/Fechar e mostrar ACK no histórico.
- Explicar Wi-Fi prioritário e contingência 4G.
- Relacionar os dados à educação sustentável e ao potencial de gamificação.

22. Limites atuais e evolução

A implementação atual é uma base funcional de IoT e aplicativo. Alguns itens do conceito completo ainda dependem do protótipo mecânico e de validação física.

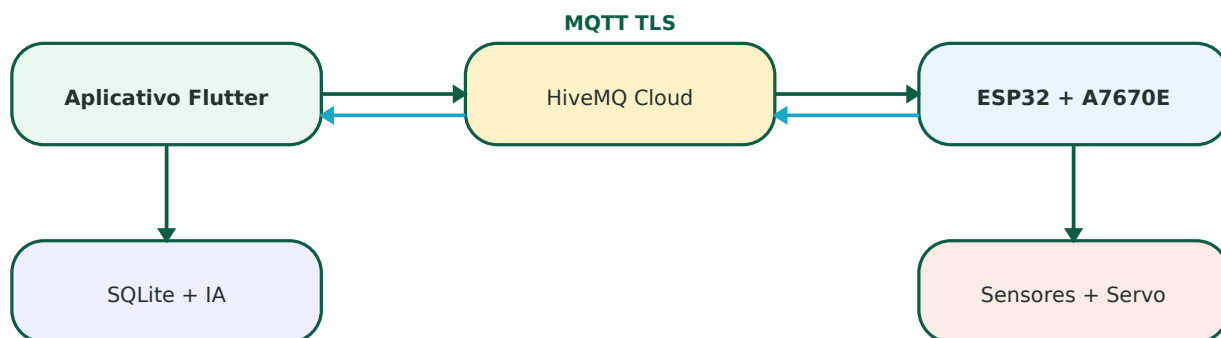
Limite atual	Evolução possível
Image Labeling genérico	modelo TFLite/LiteRT específico de resíduos
Uma comporta	roteador multiposição para papel/plástico/metal/vidro
Nível único	sensor por compartimento
Sem identidade do usuário	QR/RFID/NFC e ranking por turma
Histórico local	painel web e sincronização em nuvem
Sem ensaio mecânico automatizado	jam detection e testes de ciclo

É necessária calibração no hardware real para HX711, nível e ângulos do servo. O ECHO de sensores ultrassônicos de 5 V deve ser adequado a 3,3 V antes do ESP32.

23. Conclusão

A Lixeira Inteligente 1.1.1+5 forma um sistema integrado entre sensores, ESP32, Wi-Fi/4G, HiveMQ, aplicativo móvel, banco local e visão computacional. O firmware é responsável pela camada física e de conectividade; o aplicativo concentra monitoramento, configuração, IA, histórico e interação humana.

A separação de responsabilidades permite evoluir a proposta original de feira sem reescrever todo o sistema. O ponto central para as próximas etapas é transformar a base atual em um protótipo mecânico de múltiplos compartimentos e validar a classificação em resíduos reais.



Resumo da arquitetura implementada.

Relação com a proposta escolar

O projeto original propõe uma única abertura, identificação automática, sensores, câmera + IA, ESP32, mecanismo de desvio, registro dos descartes e acompanhamento por aplicativo. A versão documentada implementa a infraestrutura digital necessária para essa visão e deixa explícitos os pontos mecânicos e científicos que ainda precisam de validação física.

Documento técnico preparado a partir do código da versão 1.1.1+5 e do projeto escolar da Lixeira Inteligente.